

CRM LiVrai — architecture cible

Conception de la refonte du CRM de gestion des livraisons :
objectifs, périmètre fonctionnel, architecture applicative et
opérationnelle, modèle de données.

Commanditaire Meilin, chef de projet **Fait suite à** Audit du CRM existant **Statut** Pour validation
Date Septembre 2026

SOMMAIRE

1. Contexte du projet
 - a. Besoins de LiVrai
 - b. Objectifs de refonte
 - c. Éléments hors périmètre
2. Description fonctionnelle
 - a. Liste des fonctionnalités
 - b. Fonctionnement de l'application
3. Description technique
 - a. Technologies retenues
 - b. Architecture applicative
 - c. Organisation du back-end
 - d. Organisation du front-end
 - e. Communication front / back
 - f. Architecture opérationnelle
 - g. Télémétrie et supervision
4. Modèle de données
5. Réponse aux contraintes non fonctionnelles
6. Décisions d'architecture

Contexte du projet

LiVrai exploite depuis plusieurs années une application de gestion des livraisons devenue le point de passage obligé de son activité commerciale. L'audit conduit en amont a établi que cette application ne peut pas absorber la croissance de l'entreprise. Le présent document conçoit ce qui la remplace.

a. Besoins de LiVrai

LiVrai est une société de **150 employés** proposant un service de livraison de marchandises en grande quantité pour les professionnels. Son CRM interne gère le cycle de vie des commandes de livraison, de la prise de commande à la facturation.

LE BESOIN MÉTIER : RENDRE LES CLIENTS AUTONOMES

Aujourd'hui, **chaque nouveau client passe par le service commercial** : c'est lui qui établit le contrat, crée le compte dans l'application, puis réserve les livraisons pour le compte du client. Ce fonctionnement a tenu tant que le portefeuille restait modeste. Avec la croissance de l'activité, il fait du service commercial un **goulot d'étranglement** : toute commande, même répétitive, mobilise un collaborateur interne.

Le besoin exprimé est donc un changement de modèle opératoire : **les clients doivent pouvoir créer leur compte et commander leurs livraisons eux-mêmes**. Le service commercial conserve la main sur les comptes qu'il gère déjà — la fiche de mission précise que *certaines clients continueront à être gérés par le service commercial*. Les deux voies coexistent, elles ne se remplacent pas.

CE QUE CE BESOIN IMPLIQUE POUR L'ARCHITECTURE

Une application jusqu'ici réservée à un usage interne s'ouvre à des utilisateurs externes qui s'inscrivent eux-mêmes. Trois conséquences directes : le modèle d'autorisation doit être explicite et vérifié côté serveur, l'interface doit être utilisable sans formation, et la charge devient subie plutôt que maîtrisée — elle ne dépend plus du nombre d'employés mais du nombre de clients.

LE BESOIN TECHNIQUE : CE QUE L'AUDIT A ÉTABLI

L'audit a qualifié trois signaux exprimés par les équipes. Tous trois sont confirmés, et chacun a un mécanisme identifié :

SIGNAL	VERDICT	MÉCANISME ÉTABLI PAR L'AUDIT
Lenteurs	CONFIRMÉ	Connexions à la base ouvertes sans pool et jamais fermées (D-03) · chargement intégral des tables sans LIMIT — 15,6 Mo mesurés pour 20 004 livraisons (D-04) · rechargement complet de la page à chaque interaction (UX-01).
Maintenance difficile	CONFIRMÉ	Logique métier enfermée dans les servlets (D-01) · aucun test et code non testable (D-07) · socle sans support (D-11).
Disponibilité en pic	RISQUE AVÉRÉ	Session en mémoire interdisant la réplication (D-02) · saturation calculée à 150 connexions possibles pour 151 autorisées (D-03).

À ces trois signaux s'ajoutent des constats de sécurité que l'audit a vérifiés sur l'application en fonctionnement : **mots de passe stockés en clair** (D-05), **secrets de connexion versionnés dans le dépôt** (D-06), et une **fuite de données démontrée** — un compte client a pu lister l'ensemble des clients de LiVrai avec leurs adresses électroniques, et créer un utilisateur (D-09).

CE QUI EST CONSERVÉ DE L'EXISTANT

- Une refonte n'est pas une table rase. L'audit a identifié des acquis qui sont repris explicitement dans la cible :
- **Le périmètre fonctionnel** (F-06) — il correspond au métier, il est stable et compris des équipes. Il est étendu, non redéfini.
 - **Le modèle de données** (F-03) — normalisé, intégrité référentielle déclarée, **DECIMAL** pour les montants. Il constitue le socle du modèle cible, corrigé sur les points que l'audit a relevés.
 - **Les requêtes paramétrées** (F-02) et **l'échappement systématique dans les vues** (F-04) — deux classes de vulnérabilité déjà écartées. La cible doit garantir ces propriétés *par construction*, non par vigilance.
 - **La sécurité par défaut sur le routage** (F-05) — toute route nouvelle est protégée sauf mention contraire. Ce principe est conservé et généralisé à l'autorisation.

b. Objectifs de refonte

Les objectifs ci-dessous ne sont pas des intentions générales : chacun descend d'un constat mesuré de l'audit et se traduit par une décision d'architecture vérifiable.

OBJECTIF	CAUSE — CONSTAT D'AUDIT	RÉPONSE DANS L'ARCHITECTURE CIBLE
Disponibilité PRIORITÉ 1	D-02 — l'état d'authentification réside dans la <code>HttpSession</code> , donc en mémoire d'une instance. Ajouter un serveur déconnecte l'utilisateur en pleine saisie.	Back-end sans état : authentification par jeton JWT vérifié à chaque requête, aucune session serveur. Plusieurs instances deviennent interchangeables, la réplication horizontale devient possible (ADR-03).

Instance unique = point de défaillance unique.

Disponibilité

D-03 — connexion conservée en champ d'instance, jamais fermée, aucun pool. Mesure : **60 connexions dormantes** après une session de test ; saturation calculée à **150 pour 151 autorisées**.

Pool de connexions HikariCP dimensionné et borné, transactions délimitées par `@Transactional`, connexions restituées systématiquement. Le plafond devient un paramètre d'exploitation, non un accident ([ADR-05](#)).

Performance

PRIORITY 1

D-04 / UX-04 — `getAllDeliveries()` sans **LIMIT : 15,6 Mo et 20 006 lignes** pour 20 004 livraisons, contre 3 Ko pour 4. Dégradation linéaire sans palier, aucun index sur `status`.

Pagination obligatoire sur toute collection exposée par l'API, filtrage et tri exécutés en SQL, index portant sur les colonnes réellement utilisées. Le coût d'un écran devient indépendant de l'historique ([ADR-06](#)).

Performance

UX-01 / UX-02 — chaque action recharge la page entière ; l'administrateur perd sa position dans le tableau. Aucun retour visuel, d'où des double-clics et un risque de traitement en double.

Application monopage Angular : seules les données changent, pas la page. États de chargement explicites et boutons neutralisés pendant l'appel ([ADR-02](#)).

Maintenabilité

PRIORITY 1

D-01 — la logique métier vit dans les servlets : tri des livraisons, règle de visibilité, résolution des noms. **D-07** — aucun test, et un code non testable (DAO instanciés par `new`, aucune interface).

Architecture en couches stricte avec une **couche service** qui porte les règles métier, indépendante de HTTP et donc testable. Injection de dépendances native à Spring ([ADR-04](#)).

Évolutivité

D-11 — Java 1.6, `javax.servlet` 3.0, Tomcat 8.5 en fin de support. L'écosystème ayant migré vers `jakarta`, les bibliothèques récentes sont inaccessibles. Recrutement difficile.

Socle supporté : Java 25 LTS, Spring Boot 4.1, Angular 22. **API REST** exposée, réutilisable par une application mobile ou une intégration client sans réécrire les règles métier ([ADR-01](#)).

Sécurité

TRANSVERSE

D-05 mots de passe en clair, vérifiés en base · **D-06** secrets dans le code versionné · **D-09**

Hachage BCrypt, secrets injectés par l'environnement, **autorisation centralisée** et refus par défaut, validation déclarative des entrées aux frontières ([ADR-07](#)).

OBJECTIF	CAUSE — CONSTAT D'AUDIT	RÉPONSE DANS L'ARCHITECTURE CIBLE
	fuite de données démontrée sur /clients · D-10 volume -99 accepté et facturable.	
Portabilité	D-06 — changer d'environnement impose de recompiler . La fiche précise par ailleurs que <i>le choix de l'hébergeur n'est pas encore défini</i> .	Conteneurisation Docker de chaque composant, configuration entièrement externalisée. L'application devient indépendante de son hébergeur (ADR-08).

LE CRITÈRE DE RÉUSSITE DE LA REFONTE

Les trois risques déterminants de l'audit doivent être structurellement levés : le service doit pouvoir être **répliqué** (D-02), la saturation des connexions doit devenir un **paramètre maîtrisé** et non un seuil subi (D-03), et le coût d'affichage d'un écran doit être **indépendant du volume de l'historique** (D-04). Tout le reste en découle.

c. Éléments hors périmètre

Délimiter ce que le projet ne traite pas est une condition de sa réussite. Les éléments ci-dessous sont écartés **volontairement**, avec leur justification.

FONCTIONNALITÉS NON RETENUES

ÉLÉMENT	NATURE	JUSTIFICATION
Notifications et envoi de courriels	ÉVOLUTION POSSIBLE	Absent de la fiche de mission. Nécessiterait un service d'envoi et une gestion des files d'attente. Le modèle de données prévoit toutefois la traçabilité nécessaire pour l'ajouter ultérieurement.
Réinitialisation de mot de passe en autonomie	ÉVOLUTION POSSIBLE	Dépend de l'envoi de courriels. À la mise en service, la réinitialisation reste à la main de l'administration.
Export comptable et tableaux de bord	ÉVOLUTION POSSIBLE	Non demandé. La facturation est gérée dans l'application, mais son intégration à un outil comptable tiers est un projet distinct.
Suivi géolocalisé des livraisons	HORS SUJET	Suppose une flotte équipée et une application chauffeur. Le CRM suit des statuts, non des positions.

Affectation nominative d'un client à un commercial

ARBITRAGE ASSUMÉ

Voir l'encadré ci-dessous.

Application mobile

RENDUE POSSIBLE

Non développée, mais l'API REST est conçue pour qu'elle puisse l'être sans duplication de règles métier.

ARBITRAGE — « GÉRER CERTAINS CLIENTS »

La fiche de mission indique que le service commercial doit pouvoir « *gérer certains clients* ». Lu strictement, cela suppose une affectation nominative client → commercial, donc une table d'association, une règle d'autorisation supplémentaire et un écran d'administration des portefeuilles.

Décision : tout commercial accède à l'ensemble du portefeuille clients. LiVrai compte 150 employés, dont un service commercial de quelques personnes qui travaillent déjà en commun. Introduire une cloison de confidentialité interne à cette équipe reviendrait à payer une complexité permanente pour un problème que l'organisation actuelle ne pose pas.

Ce que cette décision coûte, et pourquoi elle reste réversible : si LiVrai structure demain une force de vente par secteur, l'affectation devra être ajoutée. Comme la règle d'accès est portée par la couche service — et non dispersée dans les contrôleurs, à la différence de l'application actuelle (D-09) — ce changement sera localisé : ni le front-end, ni le modèle de rôles ne seront touchés.

HORS PÉRIMÈTRE TECHNIQUE

- **La reprise des données existantes.** Le passage de MySQL à PostgreSQL suppose un chantier de migration à part entière : conversion des statuts français en énumération, attribution d'un rôle à chaque compte, et surtout **re-hachage impossible des mots de passe** — stockés en clair, ils ne peuvent être migrés tels quels sans reconduire la faille D-05. Tous les utilisateurs devront définir un nouveau mot de passe à la bascule. Ce chantier fait l'objet d'un document distinct ; il est signalé ici parce qu'il conditionne le jour de la mise en service.
- **Le choix de l'hébergeur et le dimensionnement de production.** La fiche indique que ce choix n'est pas arrêté. L'architecture est conçue pour être neutre vis-à-vis de l'hébergeur — c'est précisément le rôle de la conteneurisation.
- **Le test de charge de validation.** L'audit recommandait de confirmer le point de rupture sur une copie représentative de la base. Cette campagne reste à mener sur la cible avant mise en service.
- **La correction de l'application existante.** L'audit a relevé deux défauts actifs en production — le bouton « Refuser » inopérant (UX-06) et l'absence de contrôle de rôle sur `/clients` (D-09). Leur correction ne dépend d'aucune décision de refonte et **ne doit pas attendre ce projet**.

Description fonctionnelle

Le périmètre cible reprend le cycle de vie d'une commande de livraison en l'étendant sur trois axes : l'autonomie du client, la gestion de la facturation, et la traçabilité des actions.

a. Liste des fonctionnalités

LES QUATRE RÔLES

L'application existante ne connaît que deux profils, distingués par un booléen `admin` (D-09). La fiche de mission en impose **trois** ; un quatrième est ajouté, et justifié ci-dessous.

RÔLE	QUI	RESPONSABILITÉS
CLIENT EXTERNE	Entreprise cliente de LiVrai	Créer son compte · gérer ses informations · commander une livraison · suivre ses livraisons · consulter son historique et ses factures
COMMERCIAL INTERNE	Service commercial	Créer et gérer les comptes clients · commander pour le compte d'un client · consulter toutes les livraisons · gérer la facturation
EXPLOITATION INTERNE	Service livraisons	Accepter / refuser une commande · faire évoluer le statut d'une livraison · consulter toutes les livraisons · accéder à la facturation
ADMIN INTERNE	Administration technique	Créer et désactiver les comptes internes · attribuer les rôles · consulter le journal des actions

POURQUOI UN QUATRIÈME RÔLE

La fiche décrit trois rôles *métier*. Mais quelqu'un doit pouvoir créer un compte commercial et attribuer les rôles — et ce n'est ni un client, ni un opérateur de livraisons, ni raisonnablement un commercial qui s'auto-promeut. Sans rôle d'administration explicite, ce pouvoir existerait quand même, mais **implicitement**, ce qui reproduirait exactement le défaut relevé par l'audit : des droits qui ne sont définis nulle part et que l'on découvre en lisant le code.

FONCTIONNALITÉS PAR DOMAINE

CONSERVÉ repris de l'existant · ADAPTÉ existant, modifié · NOUVEAU ajouté au périmètre

FONCTIONNALITÉ	STATUT	RÔLES	ORIGINE ET JUSTIFICATION
Comptes et authentification			
Connexion / déconnexion	ADAPTÉ	Tous	Existant, mais par jeton JWT et non plus par session serveur (D-02). Message d'erreur explicite en cas d'échec (UX-03).
Inscription autonome du client	NOUVEAU	CLIENT	Cœur du besoin métier : décharger le service commercial de la création des comptes.
Création d'un compte client par le commercial	CONSERVÉ	COMMERCIAL	La fiche prévoit que certains clients restent gérés par le commercial. Les deux voies coexistent.
Gestion des comptes internes et des rôles	NOUVEAU	ADMIN	Rendu nécessaire par le passage de deux à quatre rôles.
Clients			
Consulter la liste des clients	ADAPTÉ	COMMERCIAL, EXPLOITATION	Existant mais accessible à tout utilisateur authentifié — fuite de données démontrée (D-09). Désormais soumise au rôle.
Gérer les informations d'un client	NOUVEAU	COMMERCIAL, CLIENT	Exigé par la fiche. Le client gère ses propres informations, le commercial celles de tous.
Livraisons			
Commander une livraison	ADAPTÉ	CLIENT, COMMERCIAL	Existant, enrichi des adresses d'enlèvement et de livraison et d'une date souhaitée. Validation des saisies : volume -99 était accepté et facturable (D-10).
Consulter ses livraisons	ADAPTÉ	CLIENT	Existant, désormais paginé, filtrable et triable (D-04, UX-04).
Consulter toutes les livraisons	ADAPTÉ	COMMERCIAL, EXPLOITATION	Existant. Les intitulés d'écran sont adaptés au profil (UX-07).
Accepter / refuser une commande	ADAPTÉ	EXPLOITATION	Existant, mais le refus était impossible : le formulaire postait vers une route ne traitant pas les écritures, d'où un 405 (UX-06).

FONCTIONNALITÉ	STATUT	RÔLES	ORIGINE ET JUSTIFICATION
Faire évoluer le statut d'une livraison	NOUVEAU	EXPLOITATION	Le cycle existant s'arrêtait à « Acceptée » puis « Terminée ». Les états intermédiaires rendent le suivi exploitable.
Suivre l'avancement d'une livraison	NOUVEAU	CLIENT	La fiche demande explicitement « commande et suivi des livraisons ».
Historique des livraisons	NOUVEAU	Tous	Exigé par la fiche. L'audit relevait qu'aucune traçabilité n'existait — ni qui, ni quand.
Facturation			
Gestion de la facturation	NOUVEAU	COMMERCIAL, EXPLOITATION	Exigé par la fiche. L'existant se limitait à un champ prix sur la livraison, sans document ni cycle de vie.
Consulter ses factures	NOUVEAU	CLIENT	Corollaire de l'autonomie du client.

4	8	7	0
RÔLES – CONTRE 2	FONCTIONS NOUVELLES	CONSERVÉES OU ADAPTÉES	FONCTION SUPPRIMÉE

b. Fonctionnement de l'application

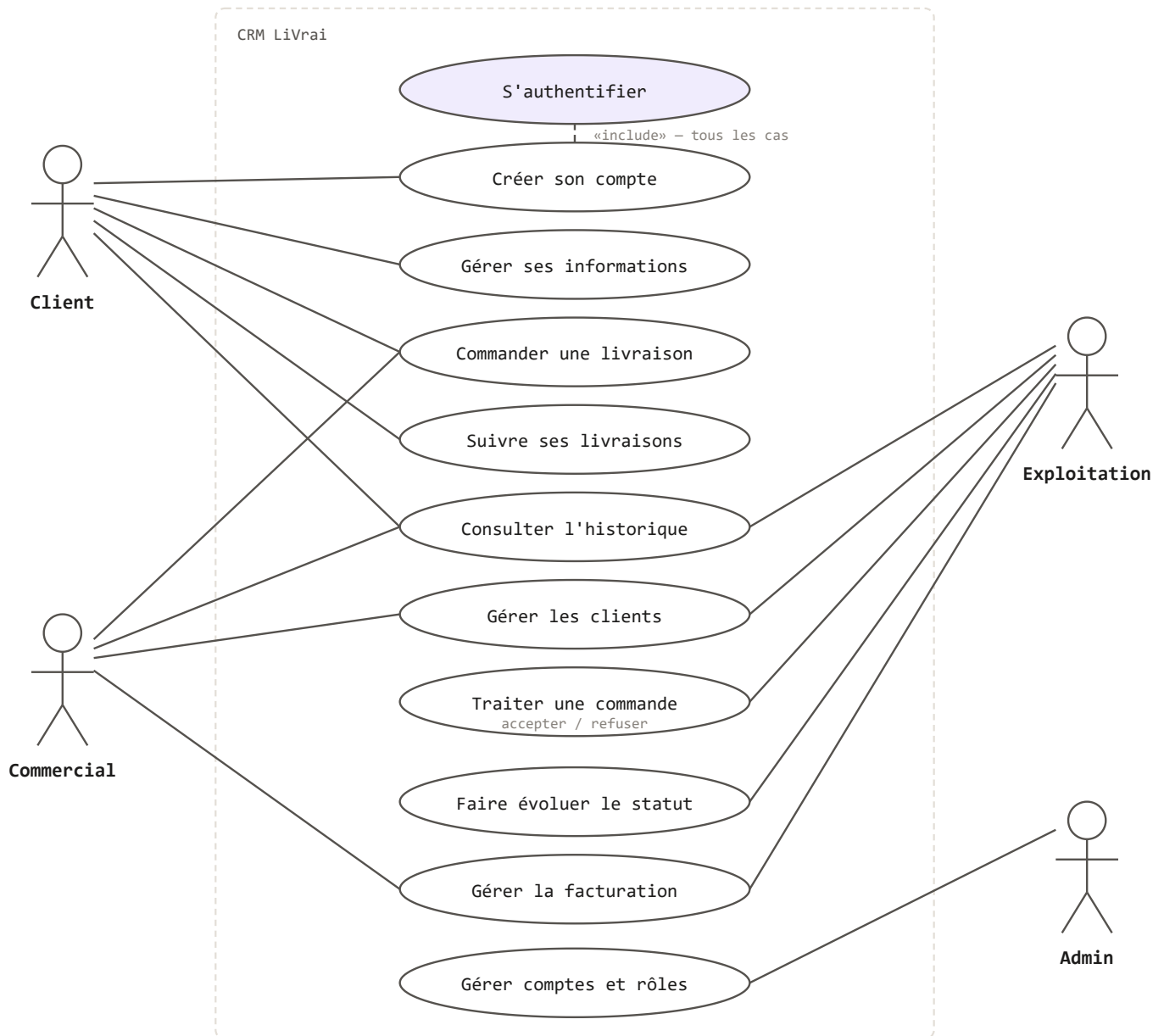
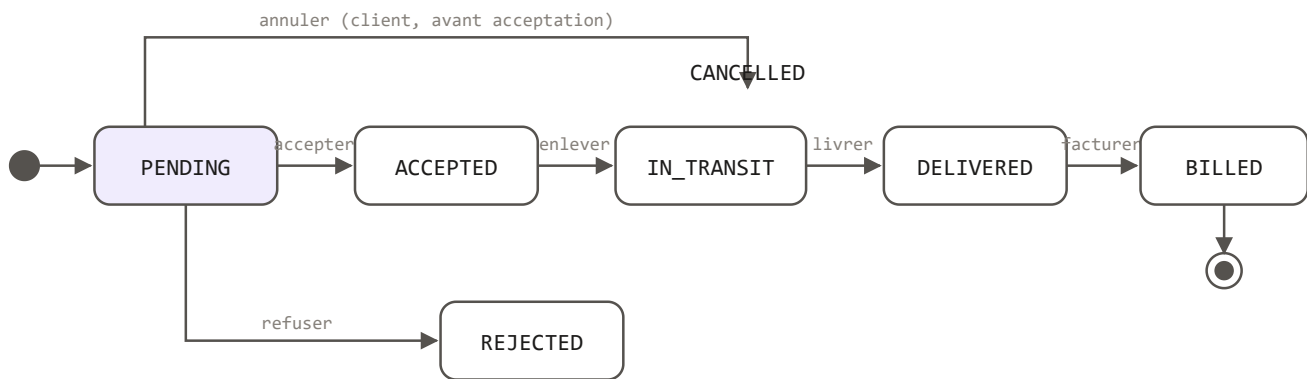


Diagramme de cas d'utilisation UML de l'application cible. Quatre acteurs au lieu de deux. Les cas « Créer son compte » (inscription autonome), « Suivre ses livraisons », « Gérer la facturation » et « Gérer comptes et rôles » sont nouveaux. Tous les cas incluent l'authentification, à l'exception de la création de compte.

CYCLE DE VIE D'UNE LIVRAISON

Le cycle existant comportait quatre états (*En attente*, *Acceptée*, *Refusée*, *Terminée*) stockés en texte libre français (D-14). La cible le précise et le formalise en énumération :



Cycle de vie d'une livraison. Les états intermédiaires **IN_TRANSIT** et **DELIVERED** sont nouveaux : ils rendent possible le suivi demandé par la fiche. Chaque transition est journalisée (qui, quand, depuis quel état), ce qu'aucune trace ne permettait auparavant.

03

Description technique

Application web à architecture trois tiers : une application monopage Angular, une API REST Spring Boot sans état, une base PostgreSQL. Chaque composant est conteneurisé.

a. Technologies retenues

COMPOSANT	TECHNOLOGIE	VERSION	JUSTIFICATION
Langage back-end	Java	25 LTS	Imposé par la fiche (« dernière version LTS »). Support jusqu'en septembre 2028 au minimum. Remplace Java 1.6, sans support depuis 2013 (D-11).
Framework back-end	Spring Boot	4.1.x	Apporte nativement ce que l'application écrivait à la main : injection de dépendances (D-07), gestion transactionnelle (D-12), sécurité centralisée (D-09), pool de connexions (D-03), configuration externalisée (D-06).
Accès aux données	Spring Data JPA / Hibernate	7.x	Requêtes paramétrées par construction — préserve F-02. Pagination native, qui est la réponse directe à D-04.
Pool de connexions	HikariCP	intégré	Réponse directe à D-03 : connexions bornées, restituées, avec délai d'attente maîtrisé.

COMPOSANT	TECHNOLOGIE	VERSION	JUSTIFICATION
Sécurité	Spring Security + JWT	7.x	Authentification sans état (D-02), autorisation déclarative et centralisée (D-09), hachage BCrypt (D-05).
Migrations de schéma	Flyway	11.x	Versionne l'évolution de la base. L'existant n'avait qu'un script d'initialisation, sans historique.
Front-end	Angular	22	Imposé par la fiche, qui justifie le choix d'une SPA par le besoin d'une « interface très interactive ». Réponse à UX-01.
État front-end	Signals Angular RETENU NgRx ÉCARTÉ	natif	Voir ADR-09 .
Base de données	PostgreSQL	18	Imposé par l'énoncé. Apporte les types énumérés (D-14), les contraintes CHECK (D-10) et les index partiels (D-04). Voir ADR-10 .
Conteneurisation	Docker / Docker Compose	27.x	Imposé par la fiche. Rend l'application indépendante de l'hébergeur, non encore choisi (D-06).
Tests	JUnit 5 · Mockito · Testcontainers · Jasmine	—	Réponse à D-07. Testcontainers permet de tester sur un vrai PostgreSQL plutôt que sur une base en mémoire au comportement différent.
Serveur web front	Nginx	1.27-alpine	Sert les fichiers statiques produits par la compilation Angular et relaie les appels API.

b. Architecture applicative

LE CHOIX DE L'ARCHITECTURE : MONOLITHE MODULAIRE

Avant de décrire l'organisation retenue, il faut justifier la famille d'architecture choisie. Trois options étaient ouvertes :

ARCHITECTURE	CE QU'ELLE APPORTERAIT À LIVRAI	CE QU'ELLE COÛTERAIT
Monolithe en couches RETENU	Séparation nette des responsabilités, testabilité, déploiement simple, réplication horizontale possible dès lors que le service est sans état.	Une seule unité de déploiement : on ne peut pas dimensionner la facturation indépendamment des livraisons. Sans importance à cette échelle.

Microservices

ÉCARTÉ

Dimensionnement et déploiement indépendants par domaine.

Réseau entre services, cohérence distribuée, observabilité distribuée, orchestration. Pour un domaine de **six entités** et une équipe réduite, le coût dépasse largement le bénéfice — et LiVrai n'a aujourd'hui **aucune équipe de plateforme**.

Architecture hexagonale stricte

PARTIELLEMENT

Découplage total du domaine vis-à-vis des technologies.

Multiplication des interfaces et des objets de transposition. On en retient le principe directeur — le domaine ne dépend pas de l'infrastructure — sans la cérémonie complète.

LE RAISONNEMENT

L'audit a montré que le problème de LiVrai n'est pas la taille de son domaine métier — treize classes, deux tables — mais l'**absence de séparation des responsabilités** et l'**état conservé en mémoire**. Un monolithe correctement découpé en couches et rendu sans état répond aux deux. Découper en microservices ajouterait une complexité distribuée sans traiter aucun des trois risques déterminants de l'audit.

Le monolithe retenu est cependant **modulaire** : le code est organisé par domaine métier (**delivery** , **invoice** , **customer** , **user**) et non par couche technique. Si un domaine devait un jour être extrait, la frontière est déjà tracée.

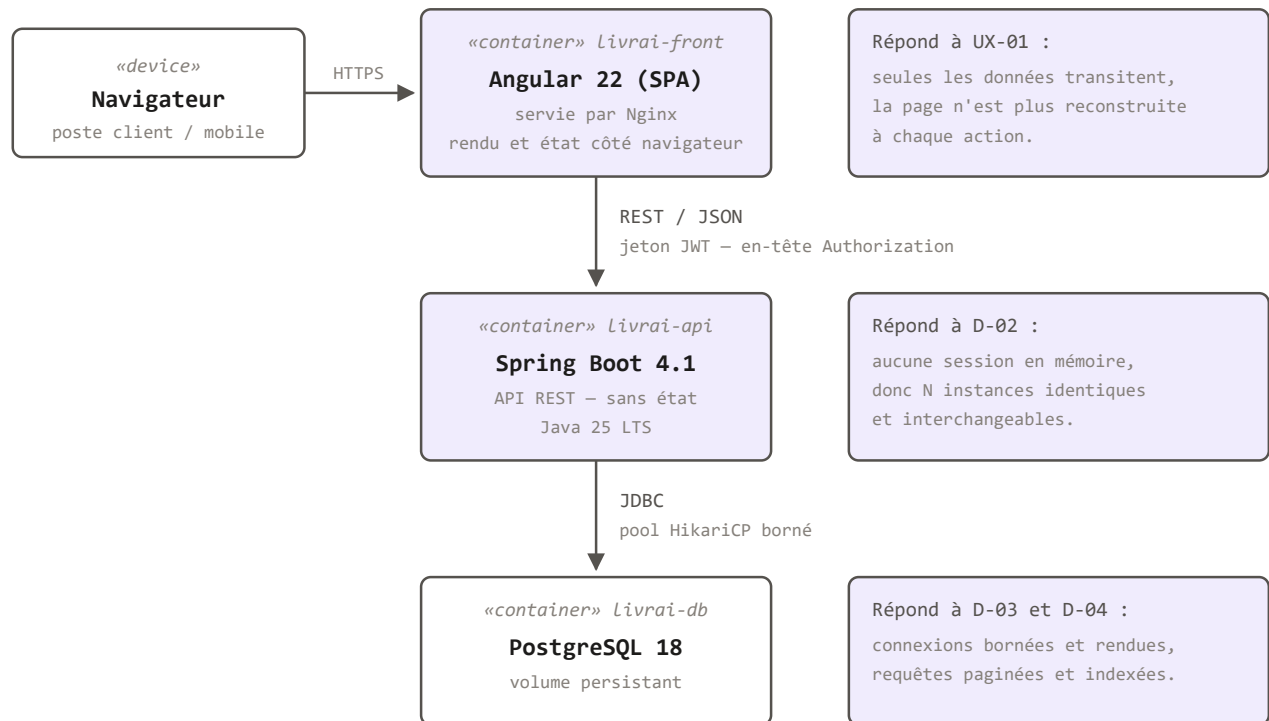
PARADIGME DE PROGRAMMATION

Le back-end suit un paradigme **orienté objet**, conformément à Java et à Spring, avec deux inflexions délibérées :

- **Programmation par contrat et immuabilité aux frontières.** Les objets de transfert (DTO) exposés par l'API sont des **record** immuables. Cela supprime une classe entière de défauts — un objet modifié en cours de traitement — et rend les données échangées prévisibles.
- **Style fonctionnel à l'intérieur des services.** L'API Stream et **Optional** remplacent les boucles et les tests de nullité qui parsèment le code actuel. Ce n'est pas une préférence esthétique : le code existant ne pouvait pas les utiliser, Java 1.6 étant antérieur à leur introduction (D-11).

Le front-end suit un paradigme **réactif et déclaratif** : l'état est représenté par des signaux, et l'interface se recalcule à partir de cet état. C'est ce qui remplace le rechargement complet de page constaté en UX-01.

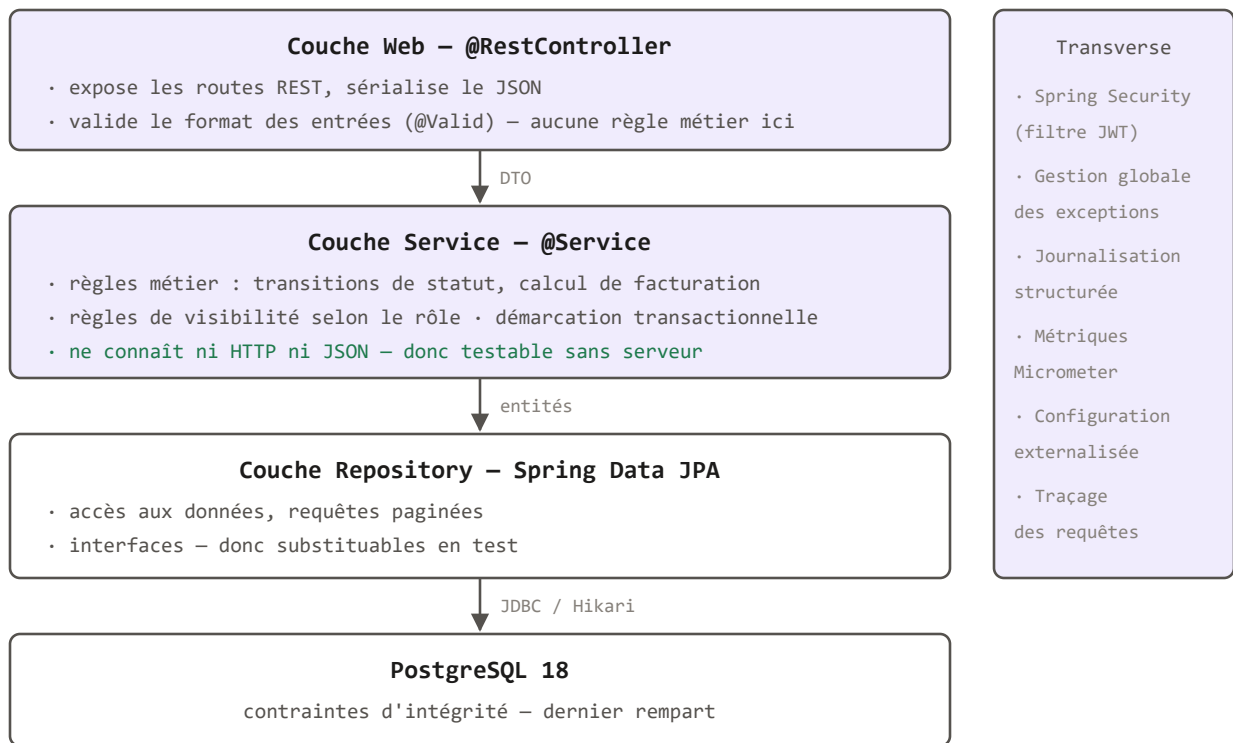
VUE D'ENSEMBLE



Vue d'ensemble en trois tiers. Chaque tier correspond à un conteneur Docker distinct. La colonne de droite rattache chaque choix au constat d'audit qui le motive.

c. Organisation du back-end

L'audit a établi que la logique métier vit dans les servlets, faute de couche dédiée (D-01), et que ce code est **structurellement non testable** : DAO instanciés par `new`, aucune interface, connexion créée dans la classe elle-même (D-07). La cible corrige les deux par une séparation stricte en quatre couches.



Architecture en couches du back-end. La couche service — absente de l'application existante — porte les règles métier et ignore HTTP, ce qui la rend testable indépendamment d'un serveur. Chaque couche ne connaît que la suivante.

ORGANISATION DU CODE PAR DOMAINE

Le découpage des paquets suit **le domaine métier**, non la couche technique. Un développeur qui travaille sur la facturation trouve tout au même endroit, plutôt que de naviguer entre un paquet `controller`, un paquet `service` et un paquet `repository` partagés :

```
fr.livrai.crm
├─ delivery/                # domaine « livraison »
│   ├── DeliveryController.java    couche web
│   ├── DeliveryService.java      règles métier
│   ├── DeliveryRepository.java    accès aux données
│   └─ Delivery.java              entité
│       └─ dto/                   objets exposés par l'API
├─ invoice/                  # domaine « facturation »
├─ customer/                 # domaine « client »
├─ user/                     # domaine « compte et rôle »
├─ security/                 # JWT, filtre, configuration d'accès
├─ common/                   # exceptions, pagination, audit
└─ config/                   # configuration Spring
```

CE QUE CE DÉCOUPAGE COÛTE

Il impose une discipline : un service ne doit pas appeler directement le repository d'un autre domaine, mais passer par le service de ce domaine. Sans cette règle, les frontières s'érodent et le bénéfice disparaît. C'est le prix à payer pour garder la possibilité d'extraire un domaine plus tard — et il doit être énoncé à l'équipe, sans quoi il ne sera pas tenu.

d. Organisation du front-end

Le front-end Angular est structuré en trois blocs aux responsabilités distinctes :

BLOC	CONTENU	RESPONSABILITÉ
core	Services HTTP, intercepteurs, gardes de route, modèles	Tout ce qui est chargé une fois et partagé : injection du jeton JWT dans chaque requête, traitement global des erreurs, contrôle d'accès aux routes selon le rôle.
features	Un dossier par domaine : deliveries, invoices, customers, admin, auth	Les écrans et leur état. Chaque domaine est chargé à la demande (lazy loading) : un client ne télécharge jamais le code de l'écran d'administration.
shared	Composants réutilisables : tableau paginé, indicateur de statut, formulaires	Homogénéité de l'interface. Le tableau paginé est le composant central : il porte la pagination, le tri et le filtrage exigés par D-04.

Le découpage par domaine est **volontairement le même** qu'au back-end. Une évolution fonctionnelle touche alors deux dossiers portant le même nom de part et d'autre, ce qui réduit le coût d'entrée dans le projet.

GESTION DE L'ÉTAT

L'état applicatif (utilisateur connecté, filtres courants, données affichées) est géré par les **signaux Angular**, natifs depuis la version 16 et stabilisés en version 22. Le choix de ne pas retenir NgRx est argumenté en [ADR-09](#).

CE QUE LE FRONT DOIT GARANTIR

- **Un retour visuel systématique** — l'audit a constaté qu'aucun indicateur de chargement n'existait, d'où des double-clics et un risque de traitement en double (UX-02). Tout appel réseau neutralise le bouton déclencheur et affiche un état de progression.
- **Des messages d'erreur explicites** — un échec de connexion réaffichait le formulaire sans explication (UX-03).
- **L'échappement du contenu dynamique** — Angular échappe par défaut, ce qui préserve la protection contre l'injection de scripts relevée comme point fort (F-04).

- **L'accessibilité** — libellés associés aux champs, ordre de tabulation, attributs ARIA. L'audit notait leur absence totale.

LE FRONT-END NE PORTE AUCUNE RÈGLE DE SÉCURITÉ

Masquer un bouton selon le rôle est un confort d'interface, **jamais une protection** : le code du navigateur est à la portée de l'utilisateur. Chaque règle d'autorisation est vérifiée côté serveur, indépendamment de ce que l'interface affiche. C'est la leçon directe de D-09, où un compte client a pu appeler une route d'administration dont le lien ne lui était pas présenté.

e. Communication front / back

Le front consomme une **API REST** exposée par le back, échangeant du JSON sur HTTPS. Le contrat est documenté automatiquement par OpenAPI.

PRINCIPES DE L'API

- **Ressources nommées au pluriel**, verbes HTTP porteurs de l'intention : `GET /api/v1/deliveries`, `POST /api/v1/deliveries`, `PATCH /api/v1/deliveries/{id}/status`.
- **Versionnement par préfixe** (`/api/v1`) — une application mobile ajoutée plus tard ne se cassera pas au premier changement de contrat.
- **Pagination obligatoire** sur toute collection. Aucune route ne retourne une liste non bornée : c'est la traduction en règle de conception du constat D-04.
- **Codes HTTP signifiants** : `200`, `201`, `400` (validation), `401` (non authentifié), `403` (non autorisé), `404`, `409` (conflit métier, par exemple une transition de statut illégale).
- **Erreurs normalisées** au format RFC 7807 *Problem Details*. L'audit relevait qu'une saisie non numérique produisait une page d'erreur brute du serveur exposée à l'utilisateur (D-10).

EXEMPLE DE CONTRAT

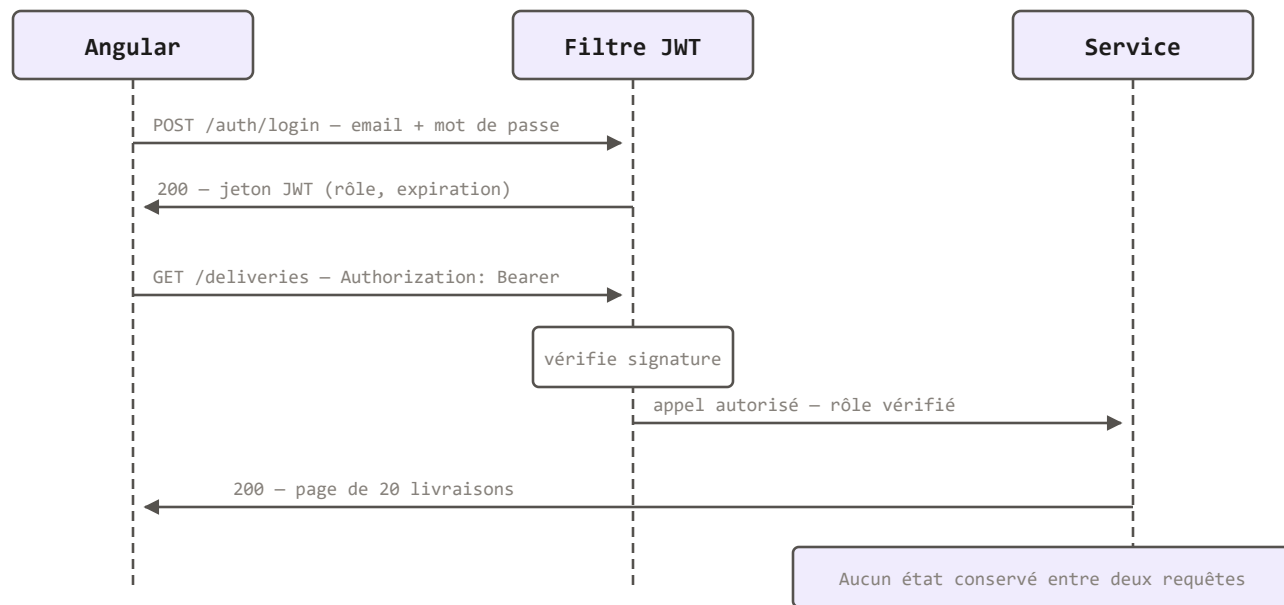
```
GET /api/v1/deliveries?status=PENDING&page=0&size=20&sort=createdAt,desc
Authorization: Bearer <jwt>
```

```
200 OK
{
  "content": [
    { "id": 42, "reference": "LIV-2026-000042", "status": "PENDING",
      "volumeM3": 2.5, "weightKg": 120.0, "customerName": "Boulangerie Durand",
      "createdAt": "2026-09-08T14:23:00Z" },
  ],
```

```
"page": { "number": 0, "size": 20, "totalElements": 1284, "totalPages": 65 }
}
```

La réponse transporte **20 lignes**, quel que soit l'historique. C'est la différence structurelle avec l'écran actuel, mesuré à 15,6 Mo pour 20 004 livraisons.

AUTHENTIFICATION ET PARCOURS D'UNE REQUÊTE



Parcours d'une requête authentifiée. Le jeton porte l'identité et le rôle ; le serveur ne conserve rien entre deux appels. C'est ce qui rend les instances interchangeables et lève le verrou D-02.

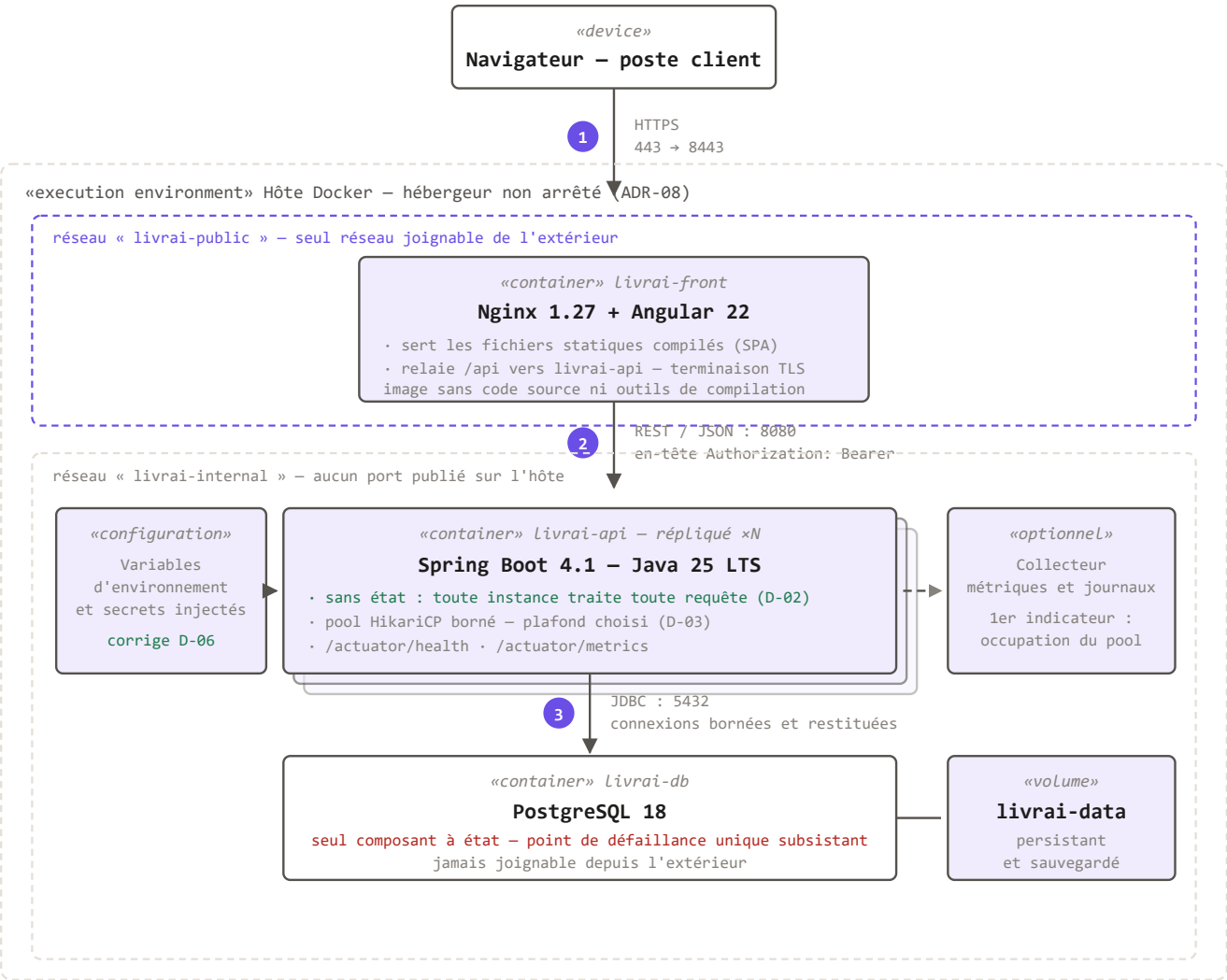
CE QUE LE CHOIX DU JWT COÛTE

Un jeton signé ne peut pas être révoqué avant son expiration : désactiver un compte ne prend effet qu'au terme de la durée de vie du jeton. C'est le prix de l'absence d'état. On le borne par une **durée de vie courte** (15 minutes) assortie d'un jeton de rafraîchissement, ce qui ramène la fenêtre d'exposition à un niveau acceptable. La solution alternative — une session partagée en Redis — supprimerait ce défaut mais réintroduirait un composant d'état à exploiter, pour un besoin que LiVrai n'a pas à cette échelle.

f. Architecture opérationnelle

La fiche précise que *le choix de l'hébergeur n'est pas encore défini* et que le déploiement se fera dans un environnement Docker. C'est exactement ce à quoi la conteneurisation répond : l'application embarque ses

dépendances et ne suppose rien de la machine qui l'exécute.



Une même image est promue de la recette à la production : seule la configuration change, jamais le binaire (D-06).
Ajouter une instance de livrai-api ne déconnecte personne – impossible dans l'application existante, dont la session résidait en mémoire (D-07).
La haute disponibilité de livrai-db dépend de l'hébergeur et n'est pas tranchée dans ce document.

Diagramme de composants UML de l'architecture opérationnelle. Trois conteneurs répartis sur deux réseaux, et le parcours numéroté d'une requête : ① le navigateur atteint le seul conteneur exposé, ② Nginx relaie vers l'API sur le réseau interne, ③ l'API interroge la base via un pool borné. Seul livrai-api est répliquable — parce qu'il ne conserve aucun état.

RÔLE DE CHAQUE CONTENEUR

CONTENEUR	IMAGE	EXPOSITION	RÔLE
livrai-front	nginx:1.27-alpine	443 (public)	Sert les fichiers statiques produits par la compilation Angular et relaie les appels /api vers l'API. Seul conteneur accessible de l'extérieur.

CONTENEUR	IMAGE	EXPOSITION	RÔLE
livrai-api	<code>eclipse-temurin:25-jre-alpine</code>	8080 (interne)	Exécute l'application Spring Boot. Répliquable à N instances : c'est le composant qui absorbe les pics de charge.
livrai-db	<code>postgres:18-alpine</code>	5432 (interne)	Base de données. Seul composant à état, adossé à un volume persistant et sauvegardé.

FLUX ENTRE CONTENEURS

1. Le navigateur appelle `livrai-front` en HTTPS et reçoit l'application Angular.
2. L'application appelle `/api/v1/...` ; Nginx relaie vers `livrai-api` sur le réseau interne.
3. `livrai-api` vérifie le jeton JWT, applique les règles d'autorisation, puis interroge `livrai-db` via le pool HikariCP.
4. La réponse JSON remonte la même chaîne. **Aucun état n'est conservé** entre deux requêtes.

DEUX RÉSEAUX, ET POURQUOI

`livrai-db` n'est rattaché qu'au réseau interne : **la base n'est pas joignable depuis l'extérieur**, même si un port était accidentellement publié sur l'hôte. Dans l'environnement de développement actuel, le port 3306 de MySQL est publié sur la machine hôte — acceptable en développement, à proscrire en production.

CONSTRUCTION DES IMAGES

Les deux images applicatives sont construites en **plusieurs étapes** : une étape de compilation (Maven pour le back, Node pour le front) puis une étape d'exécution qui ne reçoit que l'artefact produit. L'image finale ne contient ni le code source, ni les outils de compilation — elle est plus légère et expose moins de surface d'attaque.

La configuration est **intégralement externalisée** en variables d'environnement : URL de la base, identifiants, clé de signature des jetons, origines autorisées. La même image est promue de la recette à la production sans recompilation — ce que l'application actuelle interdit, ses identifiants étant codés en dur (D-06).

g. Télémétrie et supervision

L'audit relevait qu'**aucun système de journalisation n'était en place** — ni SLF4J, ni Log4j — et que les exceptions étaient soit ignorées, soit imprimées sur la sortie standard (D-08). Diagnostiquer un incident en production relevait donc de la reconstitution.

Plus important encore : la saturation démontrée par l'audit — 150 connexions pour 151 autorisées — survient **avant toute limite de processeur ou de mémoire**. Une supervision classique, qui observe la machine, ne l'aurait

pas vue venir. La télémétrie doit donc porter sur les **ressources applicatives**.

BESOIN	MOYEN	CE QUE CELA PERMET
Journaux	SLF4J + Logback, format JSON	Journaux structurés sur la sortie standard, collectés par l'orchestrateur. Chaque ligne porte un identifiant de corrélation permettant de suivre une requête de bout en bout.
État de santé	/actuator/health	Sondes de vivacité et de disponibilité. Une instance en échec est retirée de la rotation automatiquement — impossible aujourd'hui, l'instance étant unique.
Métriques	Micrometer → Prometheus	Temps de réponse par route, taux d'erreur, occupation du pool de connexions , mémoire de la JVM.
Traçage	Micrometer Tracing / OpenTelemetry	Décomposition du temps passé dans une requête. Identifie la requête SQL lente plutôt que de constater que « l'écran est lent ».
Audit métier	Table delivery_status_history	Qui a accepté, refusé ou facturé, et quand. Répond à l'absence totale de traçabilité relevée par l'audit.

LES TROIS INDICATEURS À SURVEILLER EN PRIORITÉ

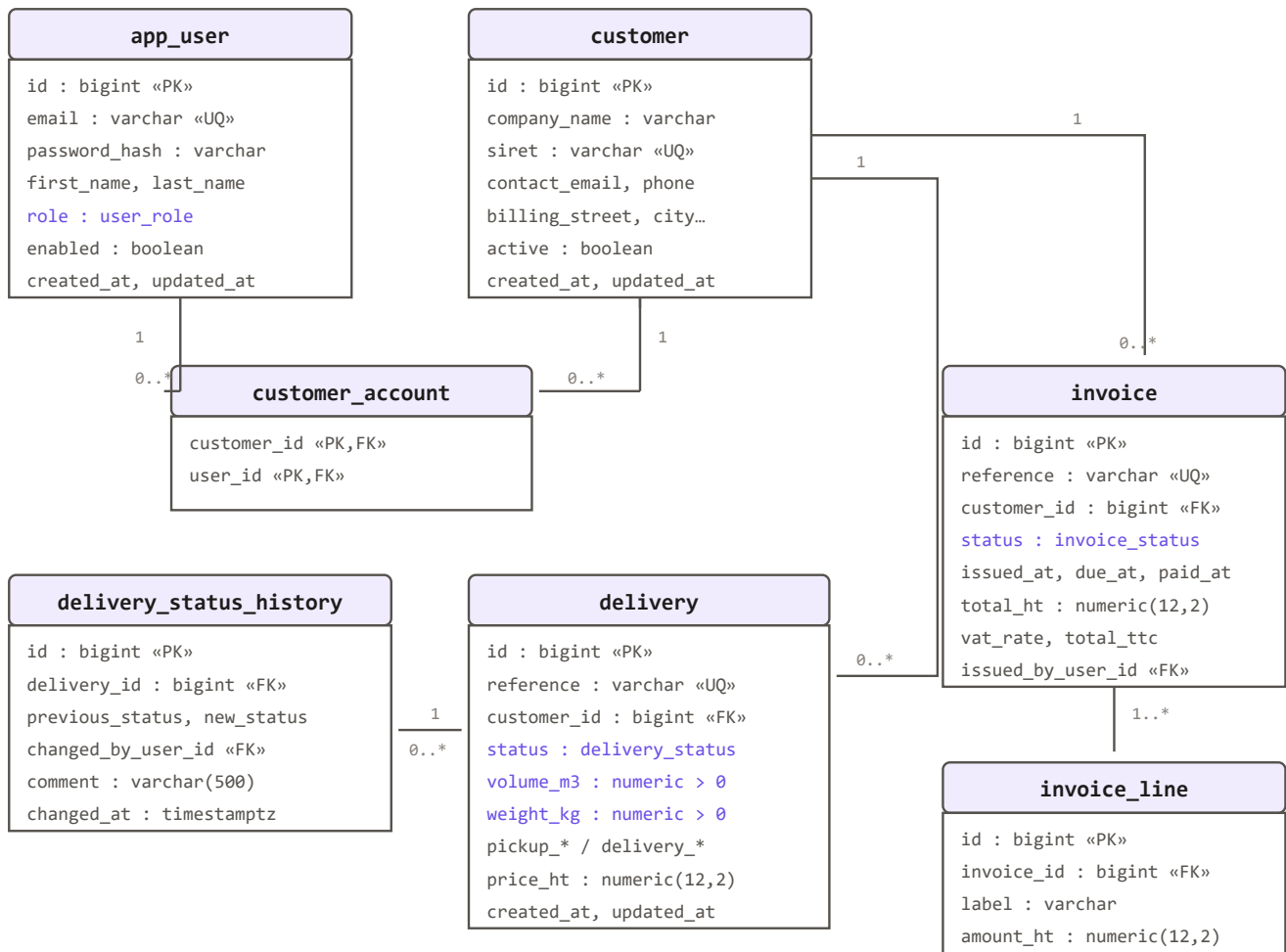
- 1. **Taux d'occupation du pool de connexions.** C'est l'indicateur qui aurait révélé D-03 avant la panne. Une alerte se déclenche à 80 % d'occupation.
- 2. **Temps de réponse du 95^e centile par route.** Détecte la dégradation progressive liée à la volumétrie (D-04) avant que les utilisateurs ne la signalent.
- 3. **Taux d'erreurs 5xx.** L'audit a constaté qu'une saisie non numérique produisait un **HTTP 500** exposé à l'utilisateur, sans qu'aucune trace n'en soit conservée (D-10).

Modèle de données

L'audit qualifiait le modèle existant de sain et directement réutilisable (F-03). Il constitue donc le point de départ — mais il ne couvre ni la facturation, ni les quatre rôles, ni la traçabilité, tous exigés par la cible.

Ce qui change, et pourquoi

EXISTANT	CIBLE	MOTIF
<code>user.admin</code> <code>BOOLEAN</code>	<code>app_user.role</code> — énumération à 4 valeurs	Un booléen ne peut pas porter quatre rôles. Cause directe de l'autorisation dispersée (D-09).
<code>user</code> confond compte et client	<code>app_user</code> + <code>customer</code> + <code>customer_account</code>	La fiche demande la gestion des informations du client. Plusieurs personnes d'une même entreprise doivent pouvoir se connecter sans dupliquer l'adresse de facturation.
<code>status</code> <code>VARCHAR(255)</code> en français	Type <code>delivery_status</code>	Une faute de frappe créait un état invalide silencieux, et la logique dépendait de la langue d'affichage (D-14).
<code>volume</code> <code>INT</code> , <code>weight</code> <code>INT</code>	<code>NUMERIC</code> avec <code>CHECK > 0</code>	Volume <code>-99</code> était accepté, stocké et facturable (D-10). Un colis peut par ailleurs mesurer 0,4 m³.
<code>price</code> sur la livraison	Tables <code>invoice</code> et <code>invoice_line</code>	La fiche exige la gestion de la facturation. Une facture regroupe plusieurs prestations, usage B2B courant. Le rattachement à la livraison est porté par le libellé et non par une clé étrangère — voir ADR-11 .
Aucune trace des actions	<code>delivery_status_history</code>	« Qui a facturé quoi, et quand » — absence relevée par l'audit. Condition de l'historique demandé par la fiche.
Index uniquement sur la clé étrangère	Index sur les colonnes réellement filtrées	Aucun index ne couvrait <code>status</code> , critère de tri principal, et le tri se faisait en mémoire Java (D-04).



Sept tables, aucun chemin circulaire. Les champs en couleur corrigent un constat d'audit.

Diagramme de classes UML du modèle cible. Les deux tables existantes deviennent sept : séparation du compte et du client, facturation, et journal des changements de statut. Les champs colorés corrigent un constat de l'audit.

Contraintes structurantes

Trois contraintes méritent d'être signalées parce qu'elles portent une règle métier au niveau du stockage, et non seulement dans le code :

- **ck_delivery_billed_has_price** — une livraison au statut **BILLED** porte obligatoirement un prix. L'état « facturée sans montant » devient impossible, quelle que soit l'erreur applicative.
- **ck_invoice_line_amount** — le montant d'une ligne de facture ne peut pas être négatif.
- **ck_delivery_volume_positive** — le volume et le poids sont strictement positifs. L'audit avait démontré qu'un volume de **-99** était accepté, stocké, affiché à l'administrateur et facturable (D-10).

VALIDATION EN DEUX ENDROITS, ET CE N'EST PAS UNE REDONDANCE INUTILE

Les mêmes règles sont exprimées dans l'application (annotations de validation) et dans la base (contraintes `CHECK`). L'application donne un message d'erreur exploitable à l'utilisateur ; la base garantit qu'aucun chemin — script de reprise, correction manuelle, défaut applicatif — ne peut créer un état invalide. Le coût est de maintenir les deux en cohérence lors d'une évolution.

Scripts SQL livrés

FICHIER	PRODUCTION	CONTENU
db/postgres/01-schema.sql	OUI	Types énumérés, sept tables, contraintes d'intégrité et de validation, index, déclencheurs de mise à jour, séquences de références métier.
db/postgres/02-seed.sql	NON	Jeu de démonstration : quatre comptes internes, trois entreprises clientes, six livraisons couvrant l'ensemble du cycle, une facture. Les mots de passe y sont publics.
Choix assumé — facture et livraison non reliées	—	<code>invoice_line</code> ne porte aucune clé étrangère vers <code>delivery</code> : la livraison facturée est désignée en clair dans le libellé. Ce choix supprime le chemin circulaire du schéma, mais retire au SGBD la capacité d'empêcher une double facturation — ce contrôle relève désormais de la couche service. Voir ADR-11 .
db/postgres/README.md	—	Mode d'exécution et correspondance entre chaque correction du schéma et le constat d'audit qui la motive.

Ces scripts posent l'état initial. En développement continu, l'évolution du schéma est prise en charge par **Flyway** : chaque changement devient un fichier de migration versionné et rejouable, là où l'existant ne disposait que d'un script d'initialisation sans historique.

05

Réponse aux contraintes non fonctionnelles

Cette section vérifie, contrainte par contrainte, que les choix d'architecture y répondent — et nomme ce qui reste à confirmer par la mesure.

Volumétrie

Constat. L'écran principal pesait 15,6 Mo pour 20 004 livraisons, contre 3 Ko pour 4. Progression linéaire, sans palier, indépendante du nombre d'utilisateurs : elle se dégrade avec le seul historique, que LiVrai accumule par construction (D-04).

Réponses.

- **Pagination imposée par le contrat d'API.** Aucune route ne retourne de liste non bornée. Une page de 20 livraisons pèse le même poids que la base contienne 1 000 ou 1 000 000 de lignes — le coût d'un écran devient constant.
- **Filtrage et tri exécutés en SQL**, avec les index correspondants. Le tri par statut se faisait en mémoire Java sur l'intégralité des lignes chargées.
- **Index partiel** `idx_delivery_pending` sur la file de traitement : l'écran le plus consulté par l'exploitation n'interroge qu'une fraction de la table.
- **Projections ciblées** : les listes ne chargent que les colonnes affichées, là où l'existant matérialisait tous les objets complets avant d'en produire le HTML.

CE QUI RESTE À VÉRIFIER

Ces réponses sont structurelles, mais leur effet chiffré doit être mesuré. L'audit recommandait déjà un test de charge sur une copie représentative de la base ; il doit être conduit sur la cible avant mise en service, avec le poids de page et le temps de réponse du 95^e centile comme indicateurs.

Disponibilité

Constat. Deux mécanismes distincts interdisaient la disponibilité en pic. L'état de session en mémoire rendait la réplication inopérante (D-02) ; la gestion des connexions fixait un plafond mesuré à 150 pour 151 autorisées, atteint **avant toute limite matérielle** (D-03).

Réponses.

- **Back-end sans état.** L'identité voyage dans le jeton, vérifié à chaque requête. Toute instance peut traiter toute requête : ajouter un serveur ne déconnecte plus personne. C'est la levée du verrou principal de l'audit.
- **Pool de connexions borné et restitué.** Le nombre maximal de connexions devient un paramètre d'exploitation, cohérent avec la configuration de PostgreSQL, au lieu d'être le produit accidentel du nombre de threads du serveur.
- **Sondes de santé.** Une instance défaillante est retirée de la rotation automatiquement — impossible aujourd'hui, l'instance étant unique.

- **Déploiement sans interruption.** Les instances étant interchangeables, elles peuvent être remplacées une à une. Aujourd'hui, tout redéploiement déconnecte l'ensemble des utilisateurs, les sessions en mémoire étant perdues.

LE POINT DE DÉFAILLANCE UNIQUE QUI SUBSISTE

La base de données reste un composant unique et à état. La haute disponibilité de PostgreSQL — réplication, bascule automatique — n'est pas traitée dans ce document : elle dépend de l'hébergeur, non encore choisi. **C'est une décision à prendre au moment de ce choix**, et elle doit être posée explicitement plutôt que découverte en production. À court terme, la sauvegarde régulière du volume et un temps de restauration mesuré constituent le minimum.

Maintenabilité

Constat. La logique métier vit dans les servlets (D-01) ; aucun test n'existe et le code ne pourrait pas être testé en l'état (D-07). C'est le facteur principal du coût de maintenance signalé par les équipes.

Réponses.

- **Couche service isolée.** Les règles métier ignorent HTTP : elles sont testables sans serveur, et réutilisables par une future API mobile sans duplication.
- **Injection de dépendances.** Les repositories sont des interfaces, substituables par des doublures en test. L'existant instanciat ses DAO par `new`, ce qui rendait toute substitution impossible.
- **Trois niveaux de test** : unitaire sur les services (Mockito), intégration sur les repositories avec un vrai PostgreSQL (Testcontainers), bout en bout sur les routes principales.
- **Traitement des erreurs centralisé et journalisation structurée**, là où l'existant ignorait silencieusement certaines exceptions et convertissait les autres en `RuntimeException` nue (D-08).
- **Découpage par domaine** : une évolution fonctionnelle se localise dans un dossier, au lieu de traverser des paquets techniques partagés.

Évolutivité

Constat. Le socle en fin de vie enferme l'application : l'écosystème Java ayant migré vers `jakarta`, les bibliothèques récentes sont incompatibles (D-11). Exposer une API imposerait de réécrire les règles métier enfermées dans les servlets (D-01).

Réponses.

- **Socle supporté** : Java 25 LTS, Spring Boot 4.1, Angular 22, PostgreSQL 18. Chaque composant reçoit des correctifs de sécurité, et ces technologies sont couramment pratiquées — ce qui traite aussi la difficulté de

recrutement signalée.

- **API REST versionnée** : une application mobile ou une intégration client consomme le même contrat, sans réécriture ni duplication.
- **Monolithe modulaire** : les frontières de domaine sont tracées. Si la facturation devait un jour être extraite, la découpe existe déjà.
- **Migrations versionnées** : l'évolution du schéma est tracée et rejouable.

Sécurité

CONSTAT D'AUDIT	RÉPONSE DANS LA CIBLE
D-05 — mots de passe en clair, vérifiés en base	Hachage BCrypt avec sel. La comparaison est à temps constant. Le mot de passe ne circule plus dans l'application à chaque requête.
D-06 — identifiants versionnés dans le dépôt, compte <code>root</code>	Secrets injectés par l'environnement. Compte applicatif dédié, aux droits restreints. La rotation d'un mot de passe ne demande plus de recompiler.
D-09 — autorisation dispersée, fuite démontrée sur <code>/clients</code>	Autorisation déclarative et centralisée, refus par défaut : une route sans règle explicite est inaccessible. L'oubli ne peut plus produire une ouverture.
D-10 — volume <code>-99</code> accepté ; <code>abc</code> → <code>HTTP 500</code>	Validation déclarative aux frontières, contraintes <code>CHECK</code> en base, erreurs normalisées renvoyées au format <i>Problem Details</i> .
F-02 / F-04 — acquis à préserver	JPA produit des requêtes paramétrées par construction ; Angular échappe le contenu dynamique par défaut. Les deux protections deviennent structurelles plutôt que dépendantes de la vigilance.
Nouveau — ouverture aux utilisateurs externes	HTTPS, politique CORS restrictive, limitation du débit sur les routes d'authentification et d'inscription, journalisation des tentatives échouées.

06

Décisions d'architecture

Chaque décision structurante est consignée avec son contexte, son alternative écartée et son coût. Tracer le *pourquoi* évite qu'une décision soit défaite par méconnaissance de ce qu'elle protégeait.

ADR-01 Refonte complète plutôt que modernisation progressive

CONTEXTE

Java 1.6 et `javax.servlet` empêchent l'usage des bibliothèques récentes (D-11). Les trois risques déterminants sont structurels, non ponctuels.

DÉCISION

Réécrire l'application sur un socle neuf, en conservant le périmètre fonctionnel et le modèle de données comme acquis.

ALTERNATIVE ÉCARTÉE

Migration incrémentale `javax` → `jakarta` puis introduction progressive de Spring. Écartée : l'absence de tests (D-07) rend toute transformation progressive risquée, et la migration ne traiterai ni D-02 ni D-04.

COÛT ASSUMÉ

Pas de livraison intermédiaire : la valeur n'apparaît qu'à la bascule. La compacité de l'existant — 13 classes, 700 lignes (F-01) — rend ce risque acceptable.

ADR-02 Application monopage plutôt que rendu serveur

CONTEXTE

Chaque action recharge la page entière et fait perdre sa position à l'utilisateur (UX-01). La fiche impose une SPA Angular pour une « interface très interactive ».

DÉCISION

Front Angular découplé, consommant une API REST.

COÛT ASSUMÉ

Deux bases de code et deux chaînes de compilation. Référencement naturel dégradé — sans objet pour un outil interne authentifié.

ADR-03 **Authentification par jeton JWT, sans état serveur**

CONTEXTE

La session en mémoire interdit la réplication : c'est le premier risque déterminant de l'audit (D-02).

DÉCISION

Jeton signé porté par l'en-tête `Authorization`, vérifié à chaque requête. Aucune session serveur.

ALTERNATIVE ÉCARTÉE

Sessions partagées dans Redis. Écartée : réintroduit un composant à état à exploiter et à superviser, pour un besoin que LiVrai n'a pas à cette échelle.

COÛT ASSUMÉ

Un jeton ne se révoque pas avant expiration. Borné par une durée de vie de 15 minutes et un jeton de rafraîchissement.

ADR-04 **Architecture en couches avec couche service obligatoire**

CONTEXTE

La logique métier vit dans les servlets, ce qui la rend ni réutilisable ni testable (D-01, D-07).

DÉCISION

Quatre couches strictes. Un contrôleur ne contient aucune règle métier ; un service ne connaît ni HTTP ni JSON.

COÛT ASSUMÉ

Plus de classes et des objets de transposition entre couches. C'est le prix de la testabilité — précisément ce qui manque aujourd'hui.

ADR-05 **Pool de connexions borné et transactions déclaratives**

CONTEXTE

Connexions jamais fermées, 60 dormantes mesurées, saturation à 150 pour 151 (D-03). Aucune démarcation transactionnelle (D-12).

DÉCISION

HikariCP dimensionné en cohérence avec `max_connections`, transactions délimitées par `@Transactional` au niveau du service.

CONSÉQUENCE

Le plafond de connexions devient un paramètre choisi, aligné entre application et base. Les enchaînements d'écritures — facturer plusieurs livraisons — deviennent atomiques.

ADR-06 **Pagination obligatoire sur toute collection**

CONTEXTE

15,6 Mo mesurés pour 20 004 livraisons, sans palier (D-04, UX-04).

DÉCISION

Aucune route de l'API ne retourne de liste non bornée. La pagination fait partie du contrat, pas d'une optimisation ultérieure.

COÛT ASSUMÉ

Le front doit gérer la navigation entre pages et les filtres. En contrepartie, le coût d'un écran cesse de dépendre de l'historique.

ADR-07 **Autorisation centralisée, refus par défaut**

CONTEXTE

`ClientsServlet` ne vérifiait pas le rôle : un compte client a listé tous les clients et créé un utilisateur (D-09). La cause est structurelle — le contrôle reposait sur la vigilance du développeur à chaque ajout.

DÉCISION

Règles déclaratives portées par la couche service, refus par défaut : une route sans règle explicite est inaccessible. Ce principe généralise le point fort F-05 relevé sur le routage.

CONSÉQUENCE

L'oubli produit une erreur visible plutôt qu'une ouverture silencieuse. Cette classe de défaut disparaît.

ADR-08 **Conteneurisation et configuration externalisée**

CONTEXTE

Les identifiants sont codés en dur : changer d'environnement impose de recompiler (D-06). La fiche indique que l'hébergeur n'est pas choisi.

DÉCISION

Une image par composant, configuration entièrement en variables d'environnement. La même image est promue de la recette à la production.

COÛT ASSUMÉ

L'équipe doit acquérir les pratiques Docker. En contrepartie, l'environnement de développement devient reproductible en une commande.

ADR-09 Signaux Angular plutôt que NgRx

CONTEXTE

L'énoncé encourage NgRx sans l'imposer. L'état applicatif de LiVrai est modeste : utilisateur connecté, filtres, pages de résultats.

DÉCISION

Utiliser les signaux natifs d'Angular 22, avec des services d'état par domaine.

ALTERNATIVE ÉCARTÉE

NgRx. Écartée : le patron Redux impose actions, réducteurs et effets pour chaque interaction. Sur un état de cette taille, la cérémonie dépasse le bénéfice — et l'équipe n'a pas de pratique préalable du patron.

RÉVERSIBILITÉ

L'état étant déjà isolé dans des services par domaine, l'introduction de NgRx resterait localisée si la complexité le justifiait.

ADR-10 PostgreSQL plutôt que la poursuite sous MySQL

CONTEXTE

PostgreSQL est imposé par l'énoncé. L'audit notait cependant que MySQL 8 était le seul composant du socle réellement supporté : le choix mérite d'être argumenté, non subi.

CE QUE POSTGRESQL APPORTE ICI

Types énumérés natifs, qui traitent directement D-14 ; contraintes **CHECK** pleinement appliquées, qui traitent D-10 ; index partiels, qui servent la file de traitement de D-04.

COÛT ASSUMÉ

Migration de données obligatoire — hors périmètre de ce document — et montée en compétence de l'équipe sur un nouveau SGBD.

ADR-11 Facture et livraison non reliées en base

CONTEXTE

Le modèle initial reliait chaque ligne de facture à la livraison facturée, avec une contrainte d'unicité interdisant de facturer deux fois la même livraison. Ce lien créait un chemin circulaire dans le schéma : `customer → delivery → invoice_line → invoice → customer`.

DÉCISION

`invoice_line` ne porte aucune clé étrangère vers `delivery`. La livraison facturée est désignée en clair dans le libellé de la ligne.

CE QUE LA DÉCISION APPORTE

Le schéma ne comporte plus aucun chemin circulaire et se lit plus directement. La suppression d'une livraison n'est plus retenue par une facture.

COÛT ASSUMÉ

Trois conséquences à connaître. Aucune requête ne peut reconstituer les livraisons d'une facture. **Plus aucune contrainte n'empêche la double facturation** : ce contrôle, auparavant garanti par le SGBD, repose désormais sur la couche service — donc sur la vigilance du code. Et le rapprochement facture ↔ livraison s'appuie sur du texte libre que la base ne vérifie pas.

RÉVERSIBILITÉ

Réintroduire `invoice_line.delivery_id` et sa contrainte d'unicité suffirait à rétablir la garantie, au prix du retour du chemin circulaire.

Ce que ce document ne tranche pas

Trois sujets restent ouverts et devront être décidés avant la mise en service. Les nommer ici évite qu'ils soient découverts trop tard :

1. **La haute disponibilité de la base.** Elle dépend de l'hébergeur, non encore choisi. À traiter au moment de ce choix.
2. **La stratégie de reprise des données.** En particulier le fait que tous les utilisateurs devront redéfinir leur mot de passe, les mots de passe actuels étant stockés en clair et donc non migrables sans reconduire la faille.
3. **La campagne de test de charge.** Elle validera que les seuils mesurés par l'audit sont effectivement levés, et situera le nouveau point de rupture par rapport au trafic réel de LiVrai.

Références. Ce document s'appuie sur le rapport d'audit du CRM existant (docs/audit-livrai.html), dont les références D-xx , F-xx et UX-xx sont reprises telles quelles, ainsi que sur la fiche descriptive de la mission fournie par LiVrai. Les scripts de création de la base figurent dans db/postgres/ .

Limites. Les seuils chiffrés cités proviennent de l'audit, conduit sur un environnement de test alimenté par des données générées, et non de la production. L'architecture proposée lève structurellement les mécanismes identifiés ; la mesure de son effet réel suppose la campagne de charge mentionnée ci-dessus.